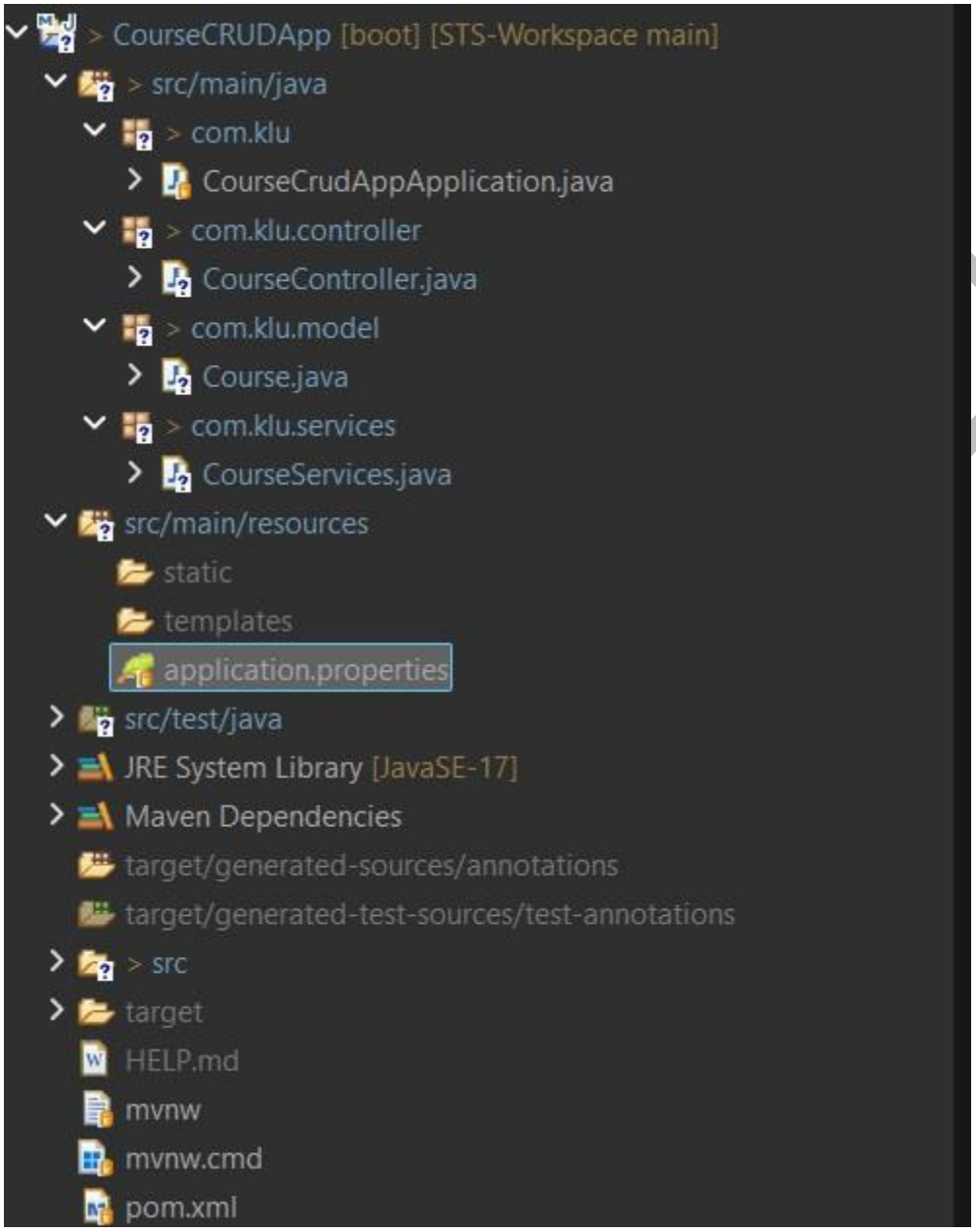
Name: R.Hemalatha

ID:2400030211

SECTION:59



CourseCRUDAppApplication.java:

```
package com.klu;


import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class CourseCrudAppApplication {

	public static void main(String[] args) {
		SpringApplication.run(CourseCrudAppApplication.class,  args);
	}

}
```

CourseController.java:

```
package com.klu.controller;

import com.klu.model.Course;
import com.klu.services.CourseServices;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/courses")
public class CourseController {

   @Autowired
   private CourseServices courseService;

   // CREATE
   @PostMapping
   public ResponseEntity<Object> addCourse(@RequestBody Course course) {
```

2400030211

```
    if (course.getTitle() == null || course.getTitle().isEmpty()) {
        return new ResponseEntity<>("Title cannot be empty", HttpStatus.BAD_REQUEST);
    }

    Course saved = courseService.addCourse(course);
    return new ResponseEntity<>(saved, HttpStatus.CREATED);
}

// READ ALL
@GetMapping
public ResponseEntity<List<Course>> getAllCourses() {
    return new ResponseEntity<>(courseService.getAllCourses(), HttpStatus.OK);
}

// READ BY ID
@GetMapping("/{id}")
public ResponseEntity<Object> getCourseById(@PathVariable int id) {

    Optional<Course> course = courseService.getCourseById(id);

    if (course.isPresent()) {
        return new ResponseEntity<>(course.get(), HttpStatus.OK);
    }

    return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
}

// UPDATE
@PutMapping("/{id}")
public ResponseEntity<Object> updateCourse(@PathVariable int id,
```

240030211

```
                                @RequestBody Course course) {

        Optional<Course> updated = courseService.updateCourse(id, course);

        if (updated.isPresent()) {
            return new ResponseEntity<>(updated.get(), HttpStatus.OK);
        }

        return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
    }

    // DELETE
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteCourse(@PathVariable int id) {

        boolean deleted = courseService.deleteCourse(id);

        if (deleted) {
            return new ResponseEntity<>("Course deleted successfully", HttpStatus.OK);
        }

        return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
    }

    // SEARCH
    @GetMapping("/search/{title}")
     public ResponseEntity<List<Course>> searchByTitle(@PathVariable String title) {
        return new ResponseEntity<>(courseService.searchByTitle(title), HttpStatus.OK);
    }
}
```

Course.java:

240003021

```java
package com.klu.model;

public class Course {

    private int courseId;
    private String title;
    private String duration;
    private double fee;

    public Course() {}

    public Course(int courseId, String title, String duration, double fee) {
        this.courseId = courseId;
        this.title = title;
        this.duration = duration;
        this.fee = fee;
    }

    public int getCourseId() { return courseId; }
    public void setCourseId(int courseId) { this.courseId = courseId; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getDuration() { return duration; }
    public void setDuration(String duration) { this.duration = duration; }

    public double getFee() { return fee; }
    public void setFee(double fee) { this.fee = fee; }
}
```

2400030211

CourseServices.java:

```
package com.klu.services;

import com.klu.model.Course;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Service
public class CourseServices {

    private List<Course> courseList = new ArrayList<>();

    // CREATE
    public Course addCourse(Course course) {
        courseList.add(course);
        return course;
    }

    // READ ALL
    public List<Course> getAllCourses() {
        return courseList;
    }

    // READ BY ID
    public Optional<Course> getCourseById(int id) {
        return courseList.stream()
                .filter(c -> c.getCourseId() == id)
                .findFirst();
```

2400030211

```
    }

    // UPDATE
    public Optional<Course> updateCourse(int id, Course updatedCourse) {
        Optional<Course> existing = getCourseById(id);

        if (existing.isPresent()) {
            Course course = existing.get();
            course.setTitle(updatedCourse.getTitle());
            course.setDuration(updatedCourse.getDuration());
            course.setFee(updatedCourse.getFee());
            return Optional.of(course);
        }

        return Optional.empty();
    }

    // DELETE
    public boolean deleteCourse(int id) {
        return courseList.removeIf(c -> c.getCourseId() == id);
    }

    // SEARCH BY TITLE
    public List<Course> searchByTitle(String title) {
        return courseList.stream()
                .filter(c -> c.getTitle().equalsIgnoreCase(title))
                .toList();
    }
}
```

OUTPUT:

240030211

http://localhost:8080/courses

Save

POST http://localhost:8080/courses Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
  "courseId": 1,
  "title": "Java",
  "duration": "3 Months",
  "fee": 10000
}
```

Body Cookies Headers (5) Test Results Status: 201 Created Time: 345 ms Size: 234 B Save Response

Pretty Raw Preview Visualize JSON

```
{
  "courseId": 1,
  "title": "Java",
  "duration": "3 Months",
  "fee": 10000.0
}
```

Pretty-print

```
{
  "courseId": 1,
  "title": "Java",
  "duration": "3 Months",
  "fee": 10000
}
```

VIVA QUESTIONS:

1. **What is ResponseEntity used for?**
   ResponseEntity → Returns data, status code, and headers together.

2. **What is the difference between @PostMapping and @PutMapping?**
   @PostMapping vs @PutMapping → POST = create, PUT = update; PUT is idempotent.

3. **Why are HTTP status codes important in REST APIs?**
   HTTP status codes → Tell client what happened (200 OK, 404 Not Found…).

4. **What happens if an invalid ID is passed to an endpoint?**
   Invalid ID → Usually returns 404 Not Found.

5. **Can ResponseEntity return both data and status codes together?**
   Data + status → ResponseEntity can return both together.

Git Hub Link : https://github.com/hemalatharachakonda/2400030211-Skill_experiments.git

2400030211